Campus Management System

ICE TASK 3

INTRODUCTION

The Campus Management System is a console-based application developed in C#. It is designed to manage important information within a campus environment, including students, lecturers, and courses. The system uses a menu-driven approach, where users interact by selecting options from a list. All data is stored in memory using lists, which keeps the program simple and easy to manage without needing a database

SYSTEM OVERVIEW

- According to the system architecture, the application is designed as a menu-driven console system.
- Users interact with the system by selecting options to add, view, and manage data.
- The system stores all data in memory using lists, making it simple and efficient

SYSTEM CONTROL

The main program controls how the system runs and how users interact with it. It uses a loop to keep the application running until the user chooses to exit.

How it works:

- Menu is displayed to the user
- User selects an option
- A switch statement calls the correct method
- Data is stored in lists (students, lecturers, courses)
- Results are displayed in the console

The system also uses **try-catch blocks** to handle invalid input and prevent crashes.

SYSTEM CONTROL (EXPANDED EXPLANATION)

The system uses a while loop to keep the program running continuously until the user decides to exit. This ensures that the user can perform multiple actions without restarting the application. Inside the loop, a switch statement is used to control which function runs based on the user's selection. This makes the system structured and easy to navigate.

Additionally, exception handling is implemented using try-catch blocks. This improves user experience by preventing the system from crashing when invalid input is entered, and instead displaying a clear error message. This contributes to a more stable and reliable system during runtime.

PERSON CLASS

At the core is the **Person class**, which acts as a base class. It contains common attributes such as :

- ID
- name
- email address

These properties use **getters and setters**, which helps control access to the data (encapsulation).

It also includes a **DisplayInfo() method**, which prints the person's details and can be reused or extended by other classes.

STUDENT CLASS

The **Student class** inherits from the Person class, meaning it automatically has access to all the attributes defined in Person. In addition to those, it includes specific properties such as :

- a student number
- the course they are enrolled in
- their marks.

The class includes a method to calculate the student's average mark:

- Uses all values in the MarksList
- Handles divide-by-zero using try-catch

It also **overrides DisplayInfo():**

- Calls base method (Person details)
- Adds student-specific details

LECTURE CLASS

The **Lecturer class** also inherits from the Person class. It includes additional attributes like :

- employee ID
- department

This inheritance structure makes the system more organized, reusable, and easier to maintain.

It overrides the **DisplayInfo() method**:

- Displays shared details from Person
- Adds lecturer-specific information

DATA HANDLING & VALIDATION

The system ensures that user input is valid before storing it.

Validation methods include:

- GetValidInt() → ensures numeric input
- GetValidString() → prevents empty input
- GetValidEmail() → checks for "@"

Additional checks:

- Prevents duplicate student IDs
- Ensures student and course exist before assignment

This improves reliability and prevents errors.

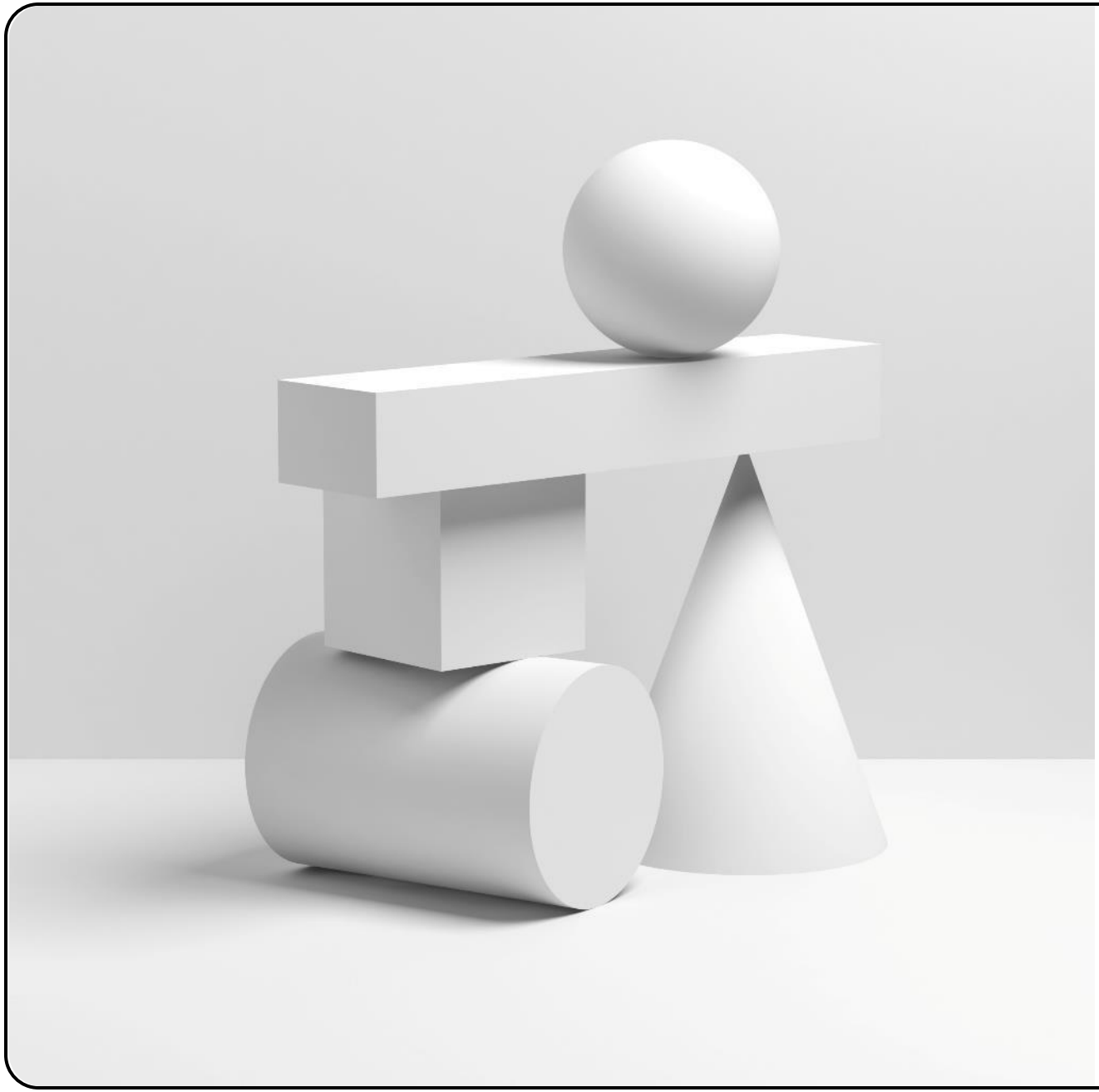
SYSTEM FEATURES

Each menu option is linked to a specific method in the program. For example, **RegisterStudent()** collects and validates user input before adding a student to the list. **AddLecturer()** performs a similar function for lecturers, while **CreateCourse()** allows new courses to be added.

The system also allows students to be assigned to courses by matching student numbers and course names. Marks can be captured and stored in a list for each student, and reports can be generated to display all stored information, including calculated averages.

CONCLUSION

Overall, the system demonstrates a practical implementation of Object-Oriented Programming. It uses inheritance to reduce code duplication, encapsulation to protect data, and polymorphism to allow flexible method behavior. The inclusion of validation and exception handling ensures that the system is robust and user-friendly, while the use of lists keeps the design simple and effective.



THANK YOU

Group members:

Musa Kekana ST10475239

Thato Esihle Mahlakanya ST10483435

Anesu Murisi ST10492047

Nomathemba Ncube ST10491804